Trouve ton artisan

Projet : Développement d’une plateforme web régionale
Nom du projet : Trouve ton artisan
Technologies : React – Node.js – Express – MySQL – Sequelize
Auteur : Maxime Gauthier
Formation : Web developpeur FullStack
Date : 27/02/2026

SOMMAIRE

- Contexte du projet
- Expression des besoins
- Contraintes techniques
- Maquettes Figma
- Architecture technique
- Base de données (MCD & MLD)
- Développement Frontend
- Développement API
- Sécurité mise en place
- Accessibilité (WCAG 2.1)
- Veille sécurité
- Déploiement
- Liens du projet

CONTEXTE DU PROJET

La région Auvergne-Rhône-Alpes couvre 12 départements et représente l'une des régions les plus artisanales de France, avec plus de 221 000 entreprises artisanales.

Dans une volonté de valorisation et de modernisation du tissu économique local, la région a souhaité développer une plateforme web permettant :

- De rechercher un artisan par catégorie
- De consulter sa fiche détaillée
- De le contacter via un formulaire sécurisé
- De mettre en avant les artisans du mois

Le projet devait respecter les normes d'accessibilité WCAG 2.1 et adopter une approche mobile-first.

EXPRESSION DES BESOINS

La plateforme devait permettre :

✔ Recherche d'artisans par catégorie
✔ Recherche par nom via barre de recherche
✔ Consultation d'une fiche artisan détaillée
✔ Envoi d'un message via formulaire de contact
✔ Mise en avant de 3 artisans "Top"
✔ Pages légales accessibles
✔ Site responsive

CONTRAINTES TECHNIQUES

Les technologies imposées étaient :

Frontend :

- ReactJS
- Bootstrap
- Sass
- Mobile First

Backend :

- Node.js
- Express
- Sequelize
- MySQL

Autres :

- Git / GitHub
- Hébergement obligatoire
- Conformité W3C
- Accessibilité WCAG 2.1

MAQUETTES FIGMA

Les maquettes ont été réalisées sur Figma pour :

- Mobile
- Tablette
- Desktop

Principes retenus :

- Interface épurée
- Hiérarchie visuelle claire
- Navigation simple
- Cohérence avec l’environnement numérique régional

https://www.figma.com/design/N0ogTwOmwaPD0SWNzBTxj0/Trouve-ton-artisan?node-id=0-1&t=gI6nFcaPVQLMtyJY-1

ARCHITECTURE TECHNIQUE

Le projet est structuré en deux parties distinctes :

Frontend (client/)

- React
- Routing avec React Router
- Service API centralisé

Backend (api/)

- Express
- Sequelize
- Routes REST
- Service d’envoi d’emails (Nodemailer)

Architecture :

Navigateur → React → API Express → MySQL

BASE DE DONNÉES

MCD - Modèle Conceptuel de Données

Entités

Catégorie

- *id* (identifiant)
- *nom*

Spécialité

- *id* (identifiant)
- *nom*

Artisan

- *id* (identifiant)
- *nom*
- *note* (sur 5)
- *ville* (localisation)
- *description* (à propos)
- *email*
- *site_web* (optionnel)
- *image* (optionnel)
- *top* (booléen)

Associations et cardinalités

- **Catégorie - Spécialité**
  - Une **catégorie** peut contenir **plusieurs spécialités**
    -> cardinalité : **Catégorie (1) -> Spécialité (0..N)**
  - Une **spécialité** appartient à **une seule catégorie**
    -> cardinalité : **Spécialité (1) -> Catégorie (1)**
- **Spécialité - Artisan**
  - Une **spécialité** peut contenir **plusieurs artisans**
    -> cardinalité : **Spécialité (1) -> Artisan (0..N)**
  - Un **artisan** appartient à **une seule spécialité**
    -> cardinalité : **Artisan (1) -> Spécialité (1)**

Ces cardinalités respectent le brief :
un artisan est rattaché à une seule spécialité et **une spécialité est rattachée à une seule catégorie**.

MLD - Modèle Logique de Données

Table categorie

- **id** (PK, INT, auto-incrément)
- **nom** (VARCHAR)

Table specialite

- **id** (PK, INT, auto-incrément)
- **nom** (VARCHAR)
- **categorie_id** (FK -> categorie.id)

Table artisan

- **id** (PK, INT, auto-incrément)
- **nom** (VARCHAR)
- **note** (DECIMAL)
- **ville** (VARCHAR)
- **description** (TEXT)
- **email** (VARCHAR)
- **site_web** (VARCHAR, nullable)
- **image** (VARCHAR, nullable)
- **top** (BOOLEAN, default FALSE)
- **specialite_id** (FK -> specialite.id)

Justification de la colonne top

Le brief demande d'afficher sur la page d'accueil **"les trois artisans du mois"**.
La colonne top (booléenne) permet d'indiquer simplement si un artisan est mis en avant :

- top = TRUE -> artisan du mois (mis en avant sur la page d'accueil)
- top = FALSE -> artisan affiché uniquement dans la liste standard

Cette solution est adaptée car il s'agit d'un **état** (un artisan peut être mis en avant ou non), et non d'une nouvelle entité nécessitant une table séparée.

DÉVELOPPEMENT FRONTEND

Le frontend a été développé en React.

Fonctionnalités principales :

✔ Header dynamique (catégories récupérées depuis l'API)
✔ Barre de recherche fonctionnelle
✔ Liste des artisans dynamique
✔ Fiche artisan dynamique
✔ Composant réutilisable ArtisanCard
✔ Hook personnalisé useSEO

Le projet respecte le principe de composants réutilisables et d'architecture modulaire.

DÉVELOPPEMENT API

L'API REST a été développée avec Express et Sequelize.

Routes principales :

GET /categories
GET /artisans
GET /artisans/:id
GET /artisans?search=
GET /artisans/top
POST /contact

L'API interagit exclusivement avec la base MySQL.

Le formulaire envoie un email réel via Brevo à l'artisan concerné.

SÉCURITÉ MISE EN PLACE

Plusieurs bonnes pratiques ont été implémentées :

1) Variables d'environnement (.env)

Mise en œuvre : les informations sensibles (connexion DB, identifiants email/Brevo, URL du front) sont stockées dans des variables d'environnement (`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `MAIL_HOST`, `MAIL_USER`, `MAIL_PASS`, etc.). Le fichier `.env` est ignoré par Git via `.gitignore`.
Intérêt : cela évite d'exposer des secrets dans le code source (et donc sur GitHub). Cela permet aussi d'avoir des configurations différentes selon l'environnement (local / production Render).

2) CORS configuré et restreint au frontend

Mise en œuvre : l'API Express autorise les requêtes uniquement depuis le domaine du frontend (Vercel), via une configuration CORS basée sur `FRONT_URL`.
Intérêt : empêche qu'un site tiers appelle librement l'API depuis un navigateur (réduit les risques d'abus côté client, ex. spam sur `/contact`). Même si ça ne remplace pas une authentification, c'est une barrière importante.

3) Validation des données côté serveur

Mise en œuvre : sur la route `POST /contact`, les champs sont vérifiés (`name`, `email`, `subject`, `message`, `artisanId`). En cas de champ manquant, l'API renvoie `400`.
Intérêt : éviter les envois incomplets, limiter les entrées malformées et réduire les risques d'attaques par données inattendues.

4) Vérification du format email

Mise en œuvre : une fonction `isEmail()` valide le format via une expression régulière avant l'envoi de mail.
Intérêt : éviter les erreurs d'envoi et limiter les contenus non conformes (ex. emails vides ou invalides).

5) Protection contre l'injection SQL via ORM (Sequelize)

Mise en œuvre : l'accès à la base se fait via Sequelize (ex. `Artisan.findByPk(artisanId)`). Sequelize génère des requêtes paramétrées et gère les échappements.
Intérêt : réduit fortement le risque d'injection SQL par rapport à des requêtes SQL construites à la main via concaténation.

6) Gestion propre des erreurs et réponses HTTP

Mise en œuvre : la route `/contact` est encapsulée dans un `try/catch`. En cas d'erreur, l'API log l'erreur côté serveur et renvoie un code `500` avec un message contrôlé.
Intérêt : éviter de "crasher" l'API, donner des retours clairs au client, et éviter d'exposer des détails internes sensibles dans la réponse.

7) Rate limiting (anti-spam / anti-bruteforce)

Mise en œuvre : `express-rate-limit` limite le nombre d'appels sur `POST /contact` (ex. 5 requêtes / 15 minutes). Au-delà, l'API renvoie un message d'erreur.
Intérêt : empêche qu'un utilisateur (ou un bot) spam l'endpoint de contact, protège l'infrastructure et réduit le risque d'abus.

8) Helmet : sécurisation des en-têtes HTTP

Mise en œuvre : `helmet` est utilisé sur Express pour activer des en-têtes de sécurité (ex. protection contre certaines attaques basées sur le navigateur, politiques de chargement, réduction de fuite d'informations).
Intérêt : améliore la "surface" de sécurité HTTP globale de l'API et applique des bonnes pratiques standards.

9) Sécurisation du service d'envoi d'email

Mise en œuvre : l'envoi d'email est fait côté serveur via un service dédié (Nodemailer + Brevo / ou API Brevo), jamais directement depuis le frontend.
Intérêt : évite d'exposer une clé SMTP/API côté client (ce serait critique), et permet de contrôler/limiter l'usage (validation + rate limit).

10) Séparation Front / Back et URL d'API via variable

Mise en œuvre : le frontend appelle l'API via `VITE_API_URL` (variable d'environnement Vercel).
Intérêt : évite de "hardcoder" une URL, facilite le déploiement, et permet de basculer entre environnements sans modifier le code.

L'API n'est pas accessible librement par d'autres domaines.

ACCESSIBILITÉ (WCAG 2.1)

Le projet respecte les principes suivants :

✔ Structure sémantique claire (h1 → h2 → h3)
✔ Un seul h1 par page
✔ Labels associés aux champs du formulaire
✔ Navigation clavier fonctionnelle (TAB)
✔ Utilisation d'éléments HTML natifs (button, a, input)
✔ Attribut alt sur toutes les images
✔ Contraste des couleurs suffisant

Le site est entièrement navigable au clavier.

VEILLE SÉCURITÉ

Une veille a été effectuée concernant :

- Injections SQL
- Attaques XSS

- Exposition des variables sensibles
- Vulnérabilités liées aux dépendances Node

Mesures prises :

- Utilisation d’un ORM (Sequelize)
- Validation des entrées
- Configuration sécurisée des headers
- Audit des dépendances npm

DÉPLOIEMENT

Le projet est déployé :

Frontend : trouve-ton-artisan-indol.vercel.app
Backend : https://trouve-ton-artisan-jgnw.onrender.com

La connexion entre front et back est configurée via variable d’environnement.

LIENS DU PROJET

Repository GitHub :
https://github.com/DaYSyDJiK/Trouve_ton_artisan

Site en ligne :
https://trouve-ton-artisan-indol.vercel.app

Conclusion

Le projet “Trouve ton artisan” répond aux besoins exprimés par la région Auvergne-Rhône-Alpes.

Il permet :

- Une recherche efficace d’artisans
- Une consultation détaillée
- Un contact sécurisé
- Une architecture moderne et maintenable

Le projet respecte les normes d’accessibilité et les bonnes pratiques de sécurité.